CLOUD / DEVOPS — PROJECT 2

Instrumented & Monitored Cloud Service

Order & Inventory API — structured logging, custom metrics, a Golden Signals dashboard, tiered alerting, and three real, investigated failure scenarios on AWS.

Adnan Nooruddin

eu-north-1 (Stockholm) · EC2 + CloudWatch · Terraform

The Assignment

Observability first, application complexity last

- Deploy a web app to EC2/ECS
- Structured JSON logs -> CloudWatch Logs, with correlation IDs
- ERROR / WARN / INFO log levels
- 5+ custom metrics, including real business metrics
- CloudWatch dashboard covering the Golden Signals
- CPU / memory / disk monitoring
- 3+ tiered CloudWatch alarms -> SNS -> email
- 3 failure scenarios, investigated and documented

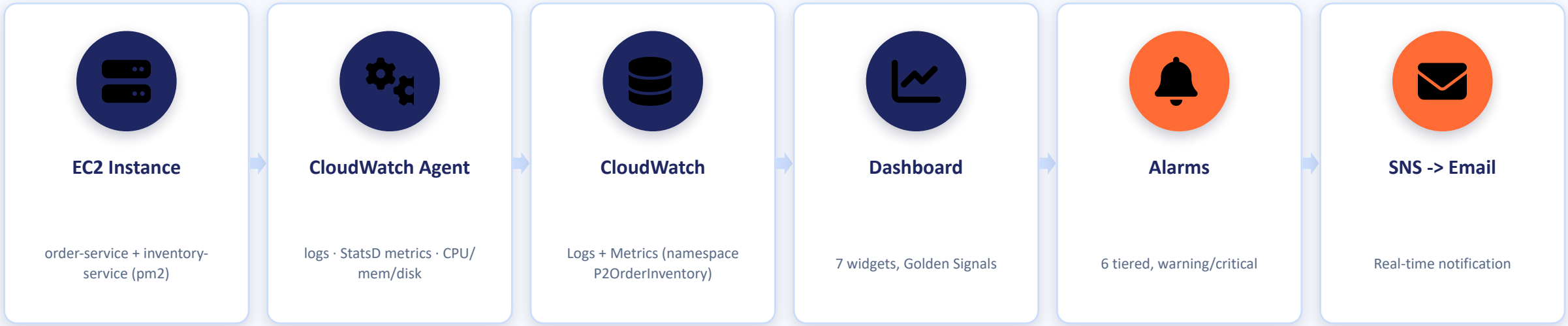
Our guiding principle

"Simple application + strong observability beats complex application + weak observability."

No database, no Docker/ECS, no load balancer, no auto-scaling — every ounce of effort went into logs, metrics, dashboard, alerting.

Architecture

One EC2 instance, one CloudWatch Agent, the full observability pipeline



t3.micro

Amazon Linux 2023

:3000 only

Public port — no SSH

SSM

Session Manager access

Terraform

EC2, IAM, SG, SNS, alarms

order-service (port 3000, public) calls inventory-service (port 3001) over localhost — never crosses the network, no security-group rule needed for it.

Instrumentation — Logs & Correlation IDs

One JSON object per line, no logging library — every field is visible in the code that emits it

Structured JSON logs

timestamp, level, service, event, correlationId + event-specific fields — queryable in CloudWatch Logs Insights

3 log levels, chosen deliberately

INFO = expected outcome, WARN = handled edge case (bad SKU), ERROR = genuine unexpected failure (downstream unreachable)

Correlation ID on every line

order-service generates it if missing, forwards X-Correlation-Id downstream; inventory-service reads or self-generates — never blank

REAL EVIDENCE — same request, both services

```
{ "service":"inventory-service",
  "event":"reservation_succeeded",
  "correlationId":"095db674-...23d8",
  "sku":"SKU-001", "remainingStock":49 }
```

```
{ "service":"order-service",
  "event":"order_created",
  "correlationId":"095db674-...23d8",
  "orderId":1, "sku":"SKU-001" }
```

Identical correlationId ties one order across both services — verified by grep, not assumed.

Instrumentation — 8 Custom Metrics

Namespace P2OrderInventory · hand-written StatsD sender, zero npm dependencies

BUSINESS METRICS

`orders_created`

Successful orders, tagged by sku — orders/minute

`order_value`

Revenue per order (USD) — Sum = revenue/period

`reservation_failures`

Failed reservations by reason + sku — surfaces which SKU is running out

`stock_level`

Live stock remaining per SKU — "about to sell out" signal

TECHNICAL METRICS

`order_errors`

Failed orders by reason (validation / rejected / unreachable)

`order_latency`

End-to-end order time incl. the inventory-service call

`reservation_attempts`

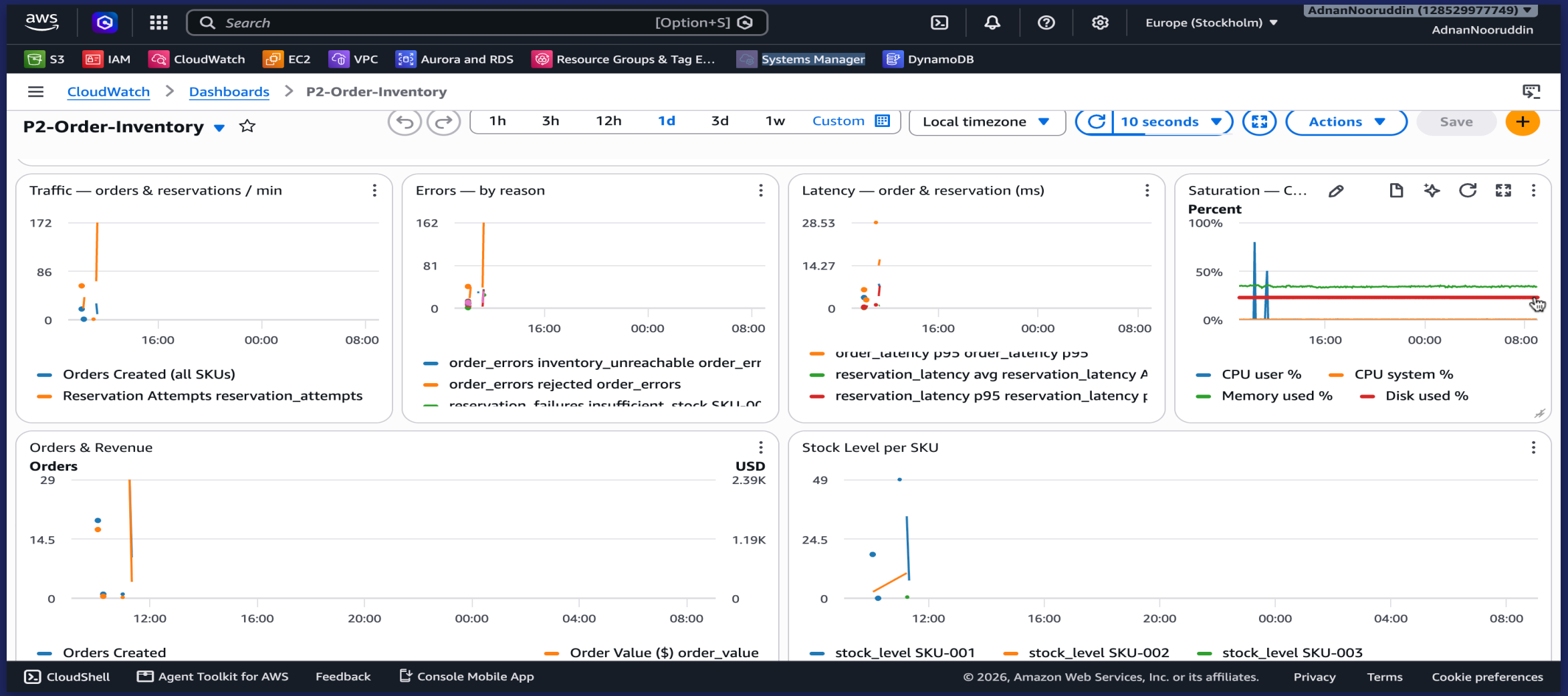
Traffic/rate signal for inventory-service

`reservation_latency`

Isolates whether slowness is order- or inventory-side

Live Demo

CloudWatch Dashboard — P2-Order-Inventory (backup capture shown below)



The Golden Signals Dashboard

7 widgets — Golden Signals across the top, business detail below

Traffic

Orders + reservation attempts / min. A flat line when load is expected means requests aren't reaching the app.

Errors

order_errors + reservation_failures, auto-broken out by reason. The specific line spiking tells you the failure type at a glance.

Latency

Average AND p95 for both services — p95 caught a real ~2-3x latency increase during CPU saturation that average alone would have hidden.

Saturation

CPU / memory / disk %. The only widget that would have caught Incident #1 — a pure-CPU event with zero log output.

Alerting — 6 Tiered Alarms

Thresholds set from real baseline data pulled from CloudWatch — not guessed

Alarm	Tier	Threshold	Observed baseline
`order-error-rate-warning`	WARNING	**> 10%**	~0% expected w/ real traffic
`order-error-rate-critical`	CRITICAL	**> 25%**	Verified: fired at 100% during Incident #2
`order-latency-warning`	WARNING	**p95 > 200ms**	avg 3.6-4.5ms, p95 4.1-6.2ms
`order-latency-critical`	CRITICAL	**p95 > 1000ms**	Never breached, even under load
`cpu-critical`	CRITICAL	**> 80%**	~0.3-0.6% — verified: fired cleanly at 100% (Incident #1)
`memory-warning`	WARNING	**> 80%**	~34-36%, very stable

All alarms: 60s period, 2-of-3 evaluation windows (avoids single-blip false alarms, still alerts within ~2-3 min) · treat_missing_data = notBreaching

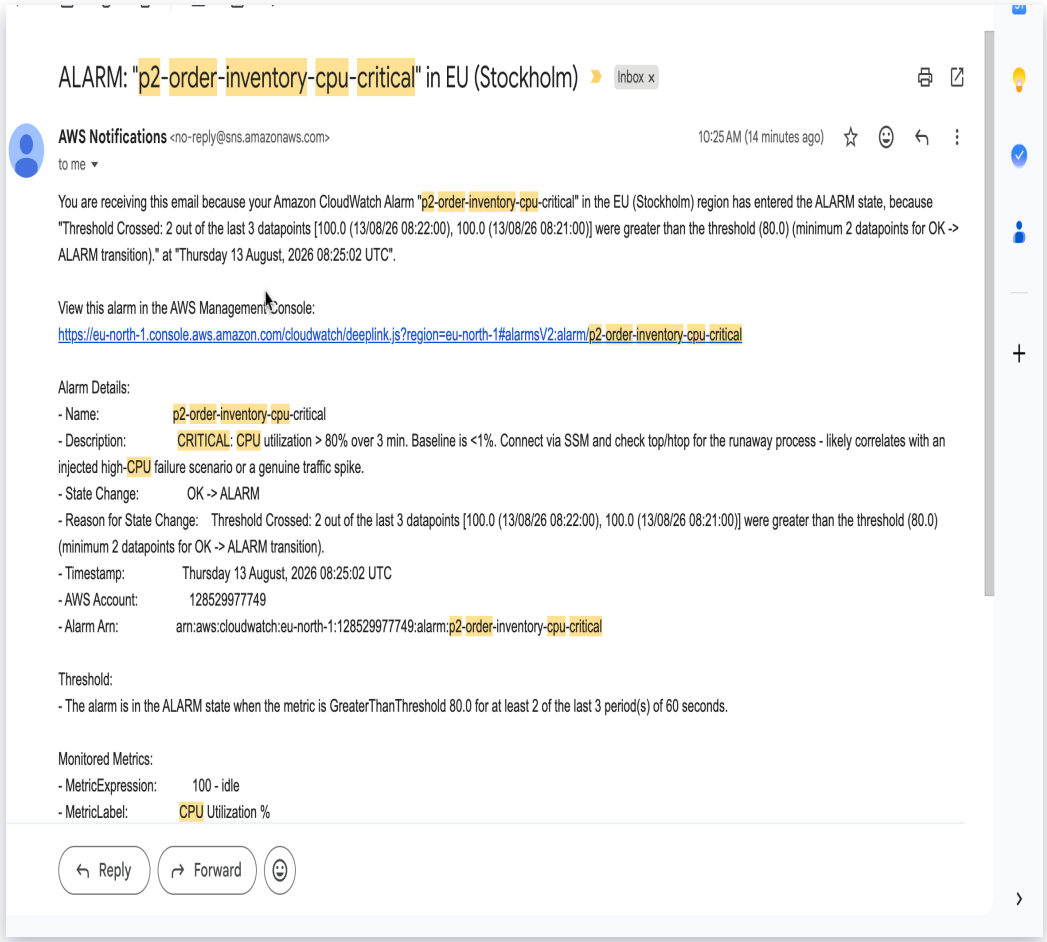
Alerting — SNS Email Flow

Verified end-to-end — real alarm, real email, not just a configured pipeline

Alarm fires → **SNS Topic** → **Email delivered**

State reverses (ALARM -> OK) -> a second email confirms recovery

```
"...cpu-critical...entered the ALARM state, because
"Threshold Crossed: 2 out of the last 3 datapoints
[100.0, 100.0] were greater than the threshold (80.0)"...'
```



ALARM: "p2-order-inventory-cpu-critical" in EU (Stockholm) Inbox x

AWS Notifications <no-reply@sns.amazonaws.com> 10:25 AM (14 minutes ago)

to me

You are receiving this email because your Amazon CloudWatch Alarm "p2-order-inventory-cpu-critical" in the EU (Stockholm) region has entered the ALARM state, because "Threshold Crossed: 2 out of the last 3 datapoints [100.0 (13/08/26 08:22:00), 100.0 (13/08/26 08:21:00)] were greater than the threshold (80.0) (minimum 2 datapoints for OK -> ALARM transition)." at "Thursday 13 August, 2026 08:25:02 UTC".

View this alarm in the AWS Management Console:
https://eu-north-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=eu-north-1#alarmsV2:alarm/p2-order-inventory-cpu-critical

Alarm Details:
- Name: p2-order-inventory-cpu-critical
- Description: CRITICAL: CPU utilization > 80% over 3 min. Baseline is <1%. Connect via SSM and check top/htop for the runaway process - likely correlates with an injected high-CPU failure scenario or a genuine traffic spike.
- State Change: OK -> ALARM
- Reason for State Change: Threshold Crossed: 2 out of the last 3 datapoints [100.0 (13/08/26 08:22:00), 100.0 (13/08/26 08:21:00)] were greater than the threshold (80.0) (minimum 2 datapoints for OK -> ALARM transition).
- Timestamp: Thursday 13 August, 2026 08:25:02 UTC
- AWS Account: 128529977749
- Alarm Arn: arn:aws:cloudwatch:eu-north-1:128529977749:alarm:p2-order-inventory-cpu-critical

Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanThreshold 80.0 for at least 2 of the last 3 period(s) of 60 seconds.

Monitored Metrics:
- MetricExpression: 100 - idle
- MetricLabel: CPU Utilization %

Reply Forward

INCIDENT #1

High CPU (Injected)

Symptom

CPU pegged at 100% for 4 min (stress-ng --cpu 2). cpu-critical alarm: OK -> ALARM at 08:25 UTC.

Investigation

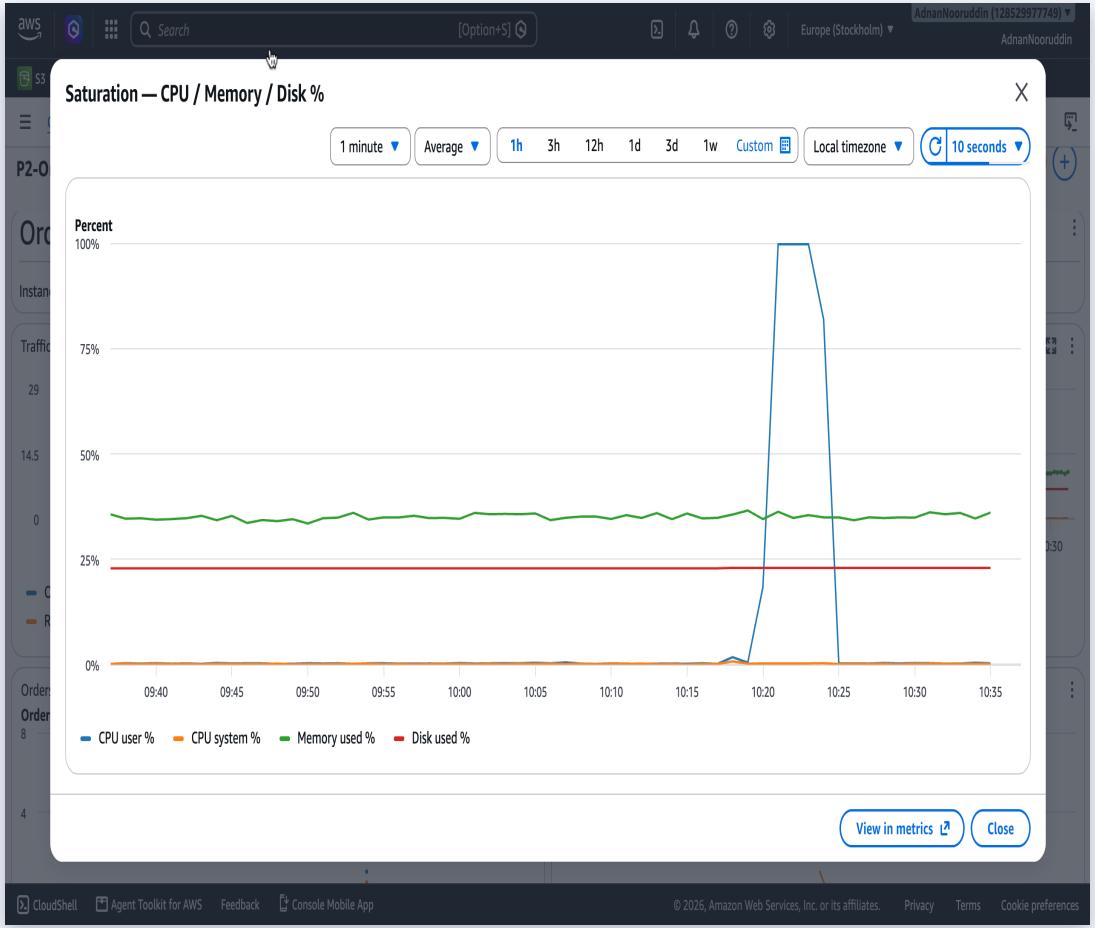
Zero application log output — a pure infrastructure event only the resource-metrics widget could surface.

Root cause & fix

Deliberate injection, self-resolving timeout. CPU + alarm both back to baseline within 4 minutes, no intervention needed.

Finding

App logs alone would have missed this entirely — confirms the value of OS-level metrics alongside application logs.



INCIDENT #2

Downstream Dependency Failure

Symptom

inventory-service stopped via pm2. 100% of orders -> 502, ERROR-level inventory_service_unreachable on every request.

Investigation

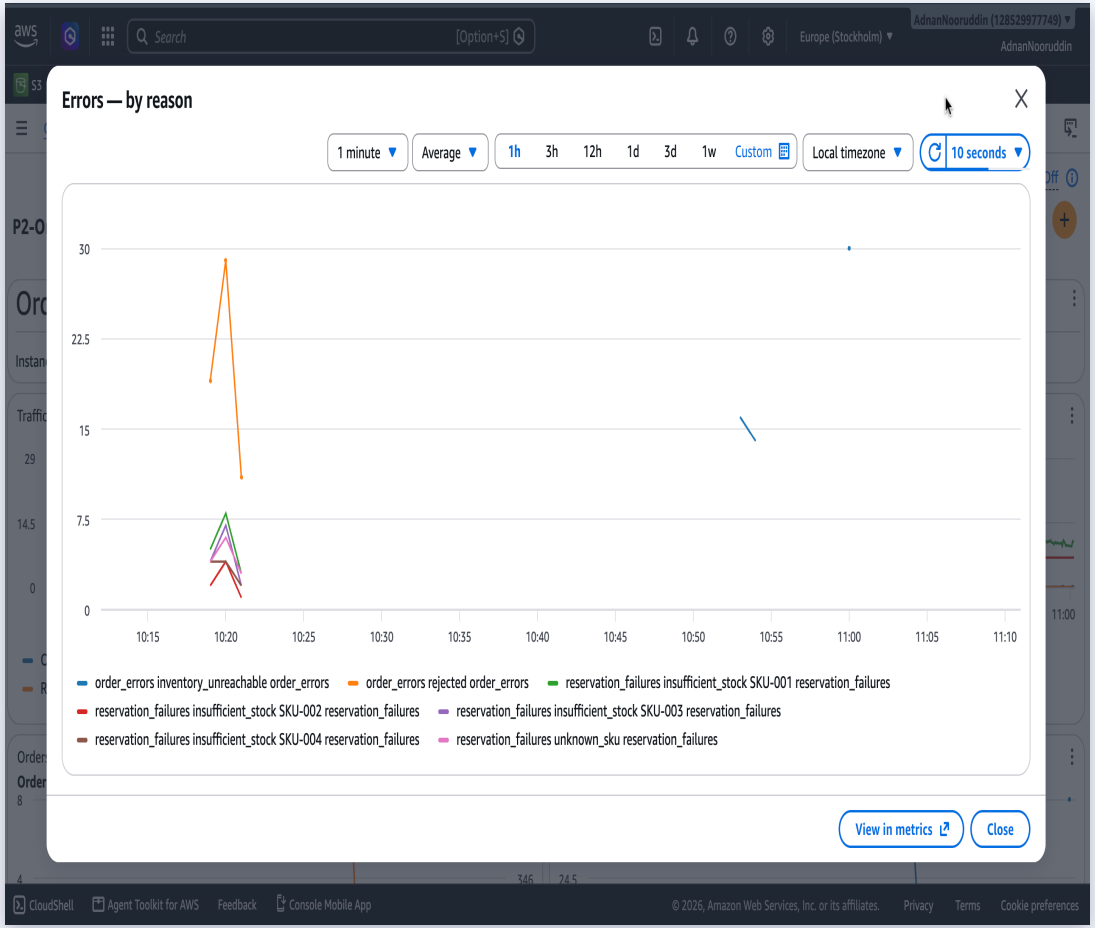
Uniform ECONNREFUSED across every request — the signature of a dependency outage, distinct from ordinary bad-SKU input.

Root cause & fix

pm2 start inventory-service. Recovery confirmed with a real successful order within ~2 minutes; both error-rate alarms cleared.

Finding

order-service has NO request timeout on the inventory-service call — a real production-readiness gap this test exposed.

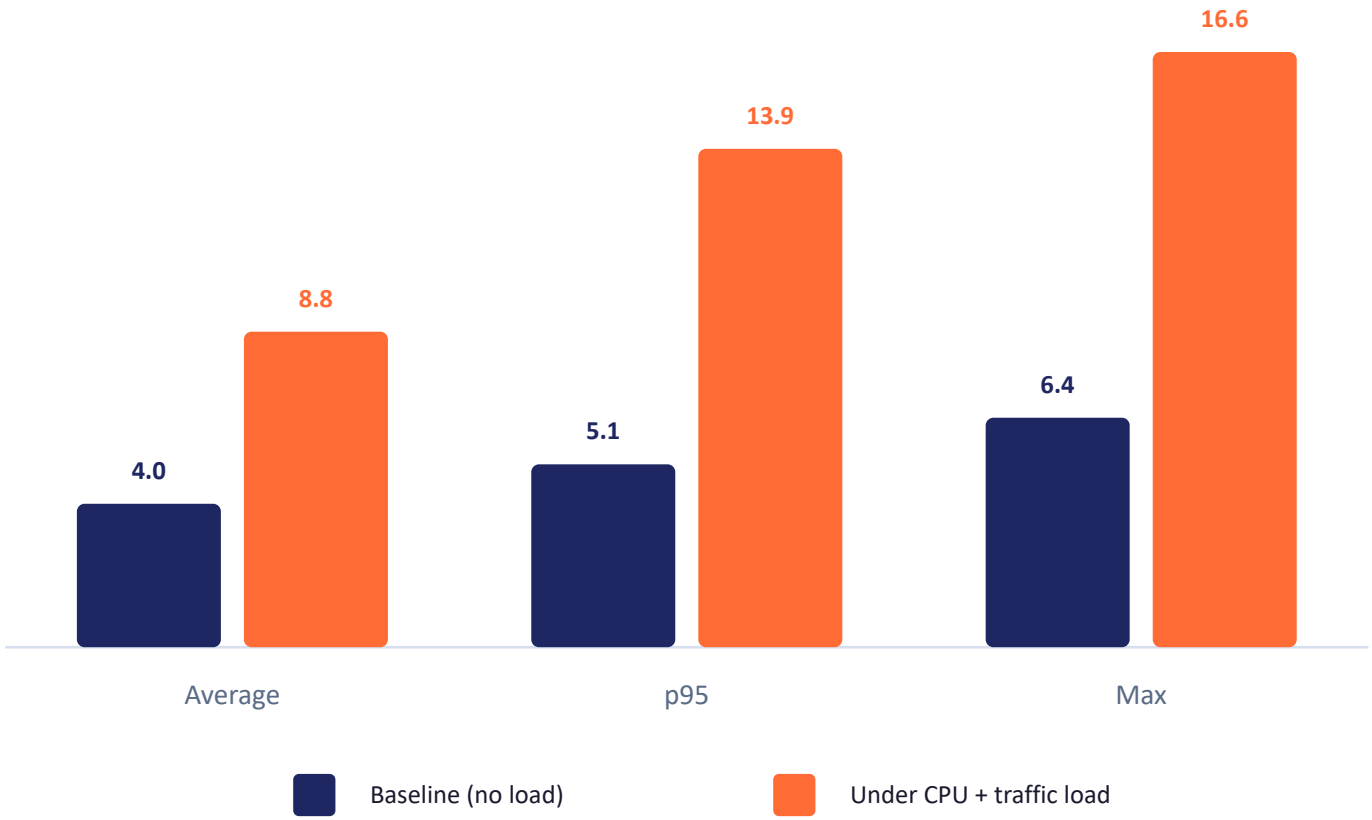


INCIDENT #3 — COMPLEX

CPU Saturation + Traffic Surge, Simultaneously

Closing the correlation question Incident #1 left open — this time load and traffic overlap on purpose

order_latency (ms) — baseline vs. under load



The real finding

~2-3x

increase in latency, directly caused by CPU contention — real, measurable, proven with overlapping data.

still 12-18x

below the 200ms warning threshold — confirming the threshold has real headroom, not miscalibrated.

Learnings & Production-Readiness Gaps

What three real, investigated incidents actually taught us

Missing request timeout

order-service's call to inventory-service has no bound — a hung (not just stopped) dependency could cascade. Found by Incident #2, not assumed.

Alarms lag reality by ~2-3 min

By design (M-of-N evaluation avoids single-blip false alarms) — but means "alarm cleared" and "problem fixed" aren't the same instant.

Which alarms fire is itself diagnostic

Saturation-only vs. errors-only vs. both pointed to three different root causes before a single log was opened.

CloudWatch Agent adds hidden dimensions

A metric_type dimension we never configured, discovered only by querying real data — led to using SEARCH() over hardcoded dashboard references.

Correlation ≠ severity

CPU contention measurably raised latency (2-3x) without being remotely alarm-worthy — proved with data instead of assumed either way.

Questions?

Structured logs · 8 custom metrics · Golden Signals dashboard · 6 tiered alarms · 3 real investigated incidents

Adnan Nooruddin · Order & Inventory Service · eu-north-1